

Axios HTTP Methods Guide

Overview

Axios is a promise-based HTTP client for making API requests. Below are the common HTTP methods and their usage patterns.

1. GET - Retrieve Data

Basic Syntax

```
axios.get(`${BASE_URL}${endpoint}`);
```

Use Cases

a) Fetch All Products (Mixed Types)

When retrieving a collection of different product types:

```
axios.get("https://localhost:3000/api/products");  
// Returns: [10 Products] (TV, Phone, Cloths, Rice, etc.)
```

b) Fetch Single Product by ID

When all products are of the same type (e.g., all restaurants) and you need a specific one:

```
axios.get("https://localhost:3000/api/products/:id");  
// Example: axios.get("https://localhost:3000/api/products/123")  
// Returns: Single product with ID 123
```

Note: Replace `:id` with the actual ID value.

c) Fetch Products by Query Parameter

When searching for products by name or other attributes:

```
axios.get("https://localhost:3000/api/products?name=productName");  
// Example: axios.get("https://localhost:3000/api/products?name=Rooftop Heaven")  
// Returns: Products matching the name "Rooftop Heaven"
```

Query Parameter Syntax: Use `?key=value` format (not `=` directly after the endpoint).

2. POST - Create New Data

Syntax

```
axios.post(URL, { data });
```

Example

```
axios.post("https://localhost:3000/api/products", {  
  name: "New Product",  
  price: 99.99,  
  category: "Electronics",  
});  
// Creates a new product with the provided data
```

Use Case

- Creating new resources (products, users, orders, etc.)
 - Submitting form data
 - Sending data to the server for processing
-

3. DELETE - Remove Data

Syntax

```
axios.delete(`${BASE_URL}${endpoint}/${id}`);
```

Example

```
axios.delete("https://localhost:3000/api/products/123");  
// Deletes the product with ID 123
```

Use Case

- Removing resources from the database
 - Deleting user accounts, products, or records
-

4. PUT - Complete Update

Syntax

```
axios.put(`${URL}/${id}`, { completeData });
```

Example

```
axios.put("https://localhost:3000/api/products/123", {  
  name: "Updated Product",  
  price: 149.99,  
  category: "Electronics",  
  description: "Fully updated product details",  
  stock: 50,  
});  
// Replaces ALL fields of product with ID 123
```

Use Case

- Complete replacement of a resource
 - All fields must be provided (missing fields may be set to null/default)
-

5. PATCH - Partial Update

Syntax

```
axios.patch(`${URL}/${id}`, { partialData });
```

Example

```
axios.patch("https://localhost:3000/api/products/123", {  
  price: 129.99,  
  stock: 45,  
});  
// Updates only the price and stock of product with ID 123  
// Other fields remain unchanged
```

Use Case

- Updating specific fields without affecting others
 - More efficient when you only need to modify a few properties
-

Key Differences: PUT vs PATCH

Method	Update Type	Fields Required	Use When
PUT	Complete	All fields	Replacing entire resource
PATCH	Partial	Only changed fields	Updating specific fields

Best Practices

1. **Error Handling:** Always use try-catch or .catch() for error handling

```
axios
  .get(url)
  .then((response) => console.log(response.data))
  .catch((error) => console.error(error));
```

2. **Async/Await:** Use modern async syntax for cleaner code

```
async function fetchProducts() {
  try {
    const response = await axios.get(url);
    return response.data;
  } catch (error) {
    console.error(error);
  }
}
```

3. **Base URL Configuration:** Set up a base URL for cleaner code

```
const api = axios.create({
  baseURL: "https://localhost:3000/api",
});

api.get("/products"); // Cleaner!
```

4. **Response Structure:** Axios responses contain:

- **data**: The response body
- **status**: HTTP status code
- **headers**: Response headers
- **config**: Request configuration

Common Status Codes

- **200 OK**: Successful GET, PUT, PATCH
- **201 Created**: Successful POST

- **204 No Content:** Successful DELETE
- **400 Bad Request:** Invalid request data
- **401 Unauthorized:** Authentication required
- **404 Not Found:** Resource doesn't exist
- **500 Internal Server Error:** Server-side error